

COE306 – Project

Wi-Fi remote-controlled robot car using ESP32

Project Overview

The project involves the development of a Wi-Fi-based controlled car using an ESP32 microcontroller. The system utilizes continuous servo motors and features a web-based interface that allows users to remotely control the robot's car. The web interface consists of five control options: left, right, backward, forward, and a stop button to manage unexpected states.

Components

- ESP32 Microcontroller
- Chassis Kit
- Continuous Servo Motors
- 3.7 Batteries
- Regulator

Wiring

ESP32	Servo Motors
Pin1	Control wire motor A
Pin 2	Control wire motor B

Implementation

Motors Testing and Calibration:

- Initially, the servo motors were tested to see their movement behaviour and response to control signals.
- Multiple calibration tests were performed using different control values to determine the operating ranges for each motor.
- It was observed that Motor A rotates in the forward direction when the control value is greater than 90 and rotates in the reverse direction when the value is less than 90.
- In contrast, Motor B rotates forward when the control value is less than 90, and rotates in reverse when the value is greater than 90.
- Both motors stop when value = 90 and the speed gets slower when the value is around 90

- Both motors stop when the control value is equal to 90. Additionally, the motor speed decreases as the control value approaches 90 from either direction.

Wi-Fi:

- The ESP32 creates its own Wi-Fi network and acts as an access point, so no external router is needed. This method was chosen because the university Wi-Fi requires an authentication step, and the mobile hotspot appears to block direct device-to-device communication, as ping test failed.
- The mobile device then connects to the ESP32 through Wi-Fi and HTTP.
- In the serial terminal, the method prints an IP address (192.169.4.1). Pasting it into the phone's browser opens the HTML page.

HTML:

- The ESP32 hosts an HTML web page that acts as the control interface for the system.
- The page contains buttons for different actions, such as moving forward, moving backward, turning, and stopping.
- Each button is linked to a specific HTTP route on the ESP32 server. When a button is pressed, the browser sends an HTTP request to that route.
- Each route is connected to a handler function in the ESP32 code. The handler function executes the required action, such as controlling the motor direction or stopping the motors.

HTML styling:

- Added a frame to contain and organize the control buttons.
- Tested the web page layout in both vertical and horizontal phone orientations.
- Rounded the edges of the buttons.

COE306 Project: Remote Car Control

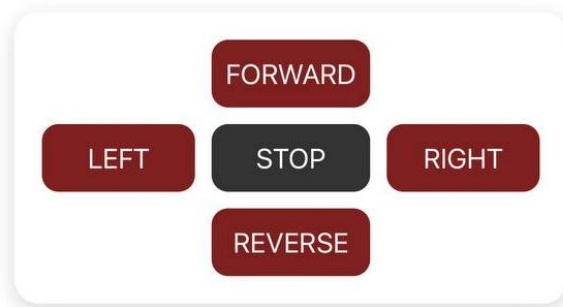


Figure 1 - Web page with five control buttons for remote car control

Power Supply:

- Two 3.7V lithium-ion batteries were used to supply the motors and the ESP32 with a regulator to drop the voltage from 8v to 5v.

Appendix

```
COE306_Project.ino
1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include <ESP32Servo.h>
4
5  // Create servo objects
6  Servo Servo_A;
7  Servo Servo_B;
8
9  // Define the GPIO pins
10 const int servoAPin = 1;
11 const int servoBPin = 2;
12
13 // Define the ESP32 access point Wi-Fi name and password
14 const char* ap_ssid = "ESP32";
15 const char* ap_password = "12345678";
16
17 // Create a web server object on port 80
18 WebServer server(80);
19
20 // This function sends the HTML page to the mobile browser
21 void handleRoot() {
22   server.send(200, "text/html",
23     "<!DOCTYPE html>"
24     "<html>"
25     "<head>"
26
27     // Makes the page responsive on mobile devices
28     "<meta name='viewport' content='width=device-width, initial-scale=1'>"
29     // CSS styling for the web page
30     "<style>"
31     "html { font-family: Helvetica; text-align: center; }"
32     "body { margin: 0; padding: 20px; }"
33     "h1 { font-size: 26px; margin-bottom: 20px; }"
```

```
COE306_Project.ino
35 //Frame that contains the control buttons
36 ".card {"
37   "max-width: 520px;"
38   "width: 100%;"
39   "box-sizing: border-box;"
40   "margin: 25px auto;"
41   "padding: 20px;"
42   "border-radius: 18px;"
43   "box-shadow: 0 4px 15px rgba(0,0,0,0.15);"
44   "background-color: white;"
45   "}"
46 //Grid layout
47 ".controlGrid {"
48   "display: grid;"
49   "grid-template-columns: repeat(3, 1fr);"
50   "gap: 12px;"
51   "align-items: center;"
52   "}"
53
54 //General button design
55 ".button {"
56   "background-color: #7F2020;"
57   "border: none;"
58   "color: white;"
59   "padding: 12px 10px;"
60   "text-decoration: none;"
61   "font-size: 20px;"
62   "cursor: pointer;"
63   "width: 100%;"
64   "border-radius: 12px;"
65   "box-sizing: border-box;"
66   "}"
67 //stop button
68 ".stop { background-color: #333333; }"
69 ".empty { visibility: hidden; }"
```

```

70     "</style>"
71
72     "<script>"
73     //send HTTP request
74     "function sendCommand(cmd){ fetch('/') + cmd); }"
75     //Only move while button is pressed
76     "function startMove(cmd){ sendCommand(cmd); }"
77     //Stop movement when the button is released
78     "function stopMove(){ sendCommand('stop'); }"
79     "</script>"
80     "</head>"
81
82     "<body>"
83     "<h1>COE306 Project: Remote Car Control</h1>"
84     //frame for styling
85     "<div class='card'>"
86     "<div class='controlGrid'>"
87     // ----- Buttons -----:
88
89     "<div></div>"
90     "<button class='button' "
91     "    onmousedown=\"startMove('forward')\" "
92     "    onmouseup=\"stopMove()\" "
93     "    ontouchstart=\"startMove('forward')\" "
94     "    ontouchend=\"stopMove()\">"
95     "    FORWARD
96     "</button>"
97     "<div></div>"
98     "<button class='button' "
99     "    onmousedown=\"startMove('left')\" "
100    "    onmouseup=\"stopMove()\" "
101    "    ontouchstart=\"startMove('left')\" "
102    "    ontouchend=\"stopMove()\">"
103    "    LEFT
104    "</button>"

```

```

105
106    "<button class='button stop' onclick=\"stopMove()\">STOP</button>"
107
108    "<button class='button' "
109    "    onmousedown=\"startMove('right')\" "
110    "    onmouseup=\"stopMove()\" "
111    "    ontouchstart=\"startMove('right')\" "
112    "    ontouchend=\"stopMove()\">"
113    "    RIGHT
114    "</button>"
115
116    "<div></div>"
117    "<button class='button' "
118    "    onmousedown=\"startMove('reverse')\" "
119    "    onmouseup=\"stopMove()\" "
120    "    ontouchstart=\"startMove('reverse')\" "
121    "    ontouchend=\"stopMove()\">"
122    "    REVERSE
123    "</button>"
124    "<div></div>"
125
126    "</div>"
127    "</div>"
128
129    "</body>"
130    "</html>"
131    );
132    }

```

COE306_Project.ino

```

135 // Handler for the forward button
136 void handleForward() {
137     Serial.println("Motor forward");
138     // Set servo directions for forward movement
139     Servo_A.write(110);
140     Servo_B.write(80);
141     // Send response to browser
142     server.send(200);
143 }
144 // Handler for the reverse button
145 void handleReverse() {
146     Serial.println("Motor reverse");
147     // Set servo directions for reverse movement
148     Servo_A.write(80);
149     Servo_B.write(110);
150     // Send response to browser
151     server.send(200);
152 }
153 // Handler for the stop button
154 void handleStop() {
155     Serial.println("Motor stop");
156     // 90 degrees is the stop value for continuous rotation servos
157     Servo_A.write(90);
158     Servo_B.write(90);
159     // Send response to browser
160     server.send(200);
161 }
162 // Handler for the right button
163 void handleRight(){
164     // Set servo directions for right turning
165     Servo_B.write(110);
166     Servo_A.write(110);
167     // Send response to browser
168     server.send(200);

```

COE306_Project.ino

```

169 }
170 // Handler for the left button
171 void handleLeft(){
172     // Set servo directions for left turning
173     Servo_A.write(70);
174     Servo_B.write(80);
175     // Send response to browser
176     server.send(200);
177 }
178
179
180 void setup() {
181     // Attach the servo objects to their GPIO pins
182     Servo_A.attach(servoAPin);
183     Servo_B.attach(servoBPin);
184
185     // Define the routes and connect each route to its handler function
186     server.on("/", handleRoot);
187     server.on("/forward", handleForward);
188     server.on("/reverse", handleReverse);
189     server.on("/stop", handleStop);
190     server.on("/left", handleLeft);
191     server.on("/right", handleRight);
192     // Start serial communication
193     Serial.begin(115200); //baud rate
194     delay(3000);
195
196     Serial.println("Program started");
197
198     // Start the ESP32 as an access point
199     Serial.println("Starting ESP32 Access Point...");
200
201     WiFi.mode(WIFI_AP);
202     WiFi.softAP(ap_ssid, ap_password);

```

```
203
204     Serial.print("ESP32 AP IP address: ");
205     Serial.println(WiFi.softAPIP());
206
207     server.on("/", handleRoot);
208     // Start the web server
209     server.begin();
210     Serial.println("Web server started");
211 }
212
213
214 void loop() {
215     // Continuously check for incoming HTTP requests from the mobile browser
216     server.handleClient();
217
218 }
```